

Preprocessing: comments to paragraph units

```
tx <- list(
  min_tokens = 60L, # a unit needs at least this many words for the topic model
  max_tokens = 150L, # a merge may not grow a unit past this; keeps topics apart
  min_sim_quantile = 0.20, # pairs less similar than this share of all neighbors
  never_merge = FALSE,
  window_n = 300L, # embedding requests per minute (limit is 500)
  window_wait = 60L, # seconds to rest between windows
  use_openrouter = TRUE, # FALSE at work
  steps = tibble::tribble(
    ~step, ~base_url_env, ~api_key_env, ~model,
    "embed", "COMPASS_EMBED_URL", "COMPASS_EMBED_KEY", "MODEL-FOR-EMBEDDING"))

suppressPackageStartupMessages({
  library(here); library(viridis)
  library(tidyverse) # tidyverse last so its verbs win
})
theme_set(theme_minimal(base_size = 12))
source(here::here("code", "llm_source.R"))
if (isTRUE(tx$use_openrouter)) {
  source(here::here("code", "llm_openrouter.R"))
  tx$steps$model <- open_router_models[tx$steps$step]
}
dir.create(here::here("outputs"), showWarnings = FALSE)
```

Corpus

Needs three columns: atom_id (comment id), Year (integer), body_english (text). The default below is a demonstration corpus that ships with quanteda; at work replace it with one line reading your file.

```
corpus_raw <- tibble(
  atom_id = quanteda::docnames(quanteda::data_corpus_inaugural),
  Year = quanteda::docvars(quanteda::data_corpus_inaugural, "Year"),
  body_english = as.character(quanteda::data_corpus_inaugural))
# At work:
# corpus_raw <- readr::read_csv(here("input", "YOUR_FILE.csv"))
```

```
miss <- setdiff(c("atom_id", "Year", "body_english"), names(corpus_raw))
if (length(miss)) stop("corpus_raw is missing: ", str_c(miss, collapse = ", "))
stopifnot("atom_id must be unique" = !anyDuplicated(corpus_raw$atom_id),
  "Year must be whole numbers" = all(corpus_raw$Year ==
```

```
round(corpus_raw$Year), na.rm = TRUE))
n0 <- nrow(corpus_raw)
corpus_raw <- dplyr::filter(corpus_raw, !is.na(Year), !is.na(body_english),
                           str_squish(body_english) != "")
writeLines(glue::glue("Kept {nrow(corpus_raw)} of {n0} comments, \\
  {min(corpus_raw$Year)} to {max(corpus_raw$Year)}."))
```

Kept 60 of 60 comments, 1789 to 2025.

From lines to paragraph units

Most comments here break after every sentence, so splitting at line breaks gives orphan sentences too short for a topic model. Three steps rebuild usable units.

1. **Split and repair.** Text is split at line breaks. A line that follows a line without end punctuation and starts lowercase is a wrapped piece of the same sentence and is joined back.
2. **Merge by meaning.** Every comment is embedded, one vector per block. Within a comment, the most similar pair of *neighboring* blocks is merged first, as long as one of them is shorter than `min_tokens`, the result stays within `max_tokens`, and the two are at least as similar as the `min_sim_quantile` point of all neighboring pairs in the corpus. This repeats until no pair qualifies. Reading order is never broken.
3. **Drop what cannot be placed.** A unit still under `min_tokens` had no neighbor it was similar enough to join without breaking the size ceiling. It is left out of the topic model rather than forced into a paragraph it would distort, and written to `dropped_units.csv` so you can read what was lost.

The two token settings are the balance you are choosing: `min_tokens` is enough text for the topic model to read, `max_tokens` is the point past which a unit starts to hold more than one subject. `topics.qmd` refits the topic model at other settings to show whether that choice drives the topics.

```
blocks <- split_blocks(corpus_raw)
tibble(measure = c("lines", "blocks after rejoining wrapped lines",
                  "median words per block", "blocks under min_tokens"),
       value = c(attr(blocks, "lines_in"), nrow(blocks),
                 stats::median(blocks$n_tokens),
                 sum(blocks$n_tokens < tx$min_tokens))) |>
tbl_paged(caption = "Blocks before merging.")
```

Blocks before merging.

measure	value
lines	1874
blocks after rejoining wrapped lines	1871
median words per block	50
blocks under min_tokens	1075

```
emb <- embed_all(blocks, p_out("embeddings.rds"), tx$steps, tx$use_openrouter,
  tx$window_n, tx$window_wait)
```

```
sims <- adjacent_sims(blocks, emb$vec)
min_sim <- stats::quantile(sims, tx$min_sim_quantile, names = FALSE)
units <- link_corpus(blocks, emb$vec, tx$min_tokens, tx$max_tokens, min_sim)
paragraphs <- dplyr::filter(units, keep)
dropped <- dplyr::filter(units, !keep)

tibble(measure = c("similarity threshold (cosine)", "units formed",
  "units kept", "units dropped", "words dropped (share)",
  "comments with nothing kept", "median words per kept unit",
  "median blocks per kept unit"),
  value = c(round(min_sim, 3), nrow(units), nrow(paragraphs), nrow(dropped),
    round(sum(dropped$n_tokens) / sum(units$n_tokens), 3),
    dplyr::n_distinct(corpus_raw$atom_id) -
    dplyr::n_distinct(paragraphs$atom_id),
    stats::median(paragraphs$n_tokens),
    stats::median(paragraphs$n_blocks))) |>
  tbl_paged(caption = "Merging results. Watch the share of words dropped
and the comments with nothing kept: if either is large, lower min_tokens or
min_sim_quantile.")
```

Merging results. Watch the share of words dropped and the comments with nothing kept: if either is large, lower min_tokens or min_sim_quantile.

measure	value
similarity threshold (cosine)	0.793
units formed	1121.000
units kept	951.000
units dropped	170.000
words dropped (share)	0.030
comments with nothing kept	0.000
median words per kept unit	125.000
median blocks per kept unit	1.000

```
ggplot(paragraphs, aes(n_tokens)) +
  geom_histogram(bins = 40, fill = viridis(1, begin = 0.35)) +
  geom_vline(xintercept = c(tx$min_tokens, tx$max_tokens), linetype = 2) +
  labs(x = "words per unit", y = "units")
```

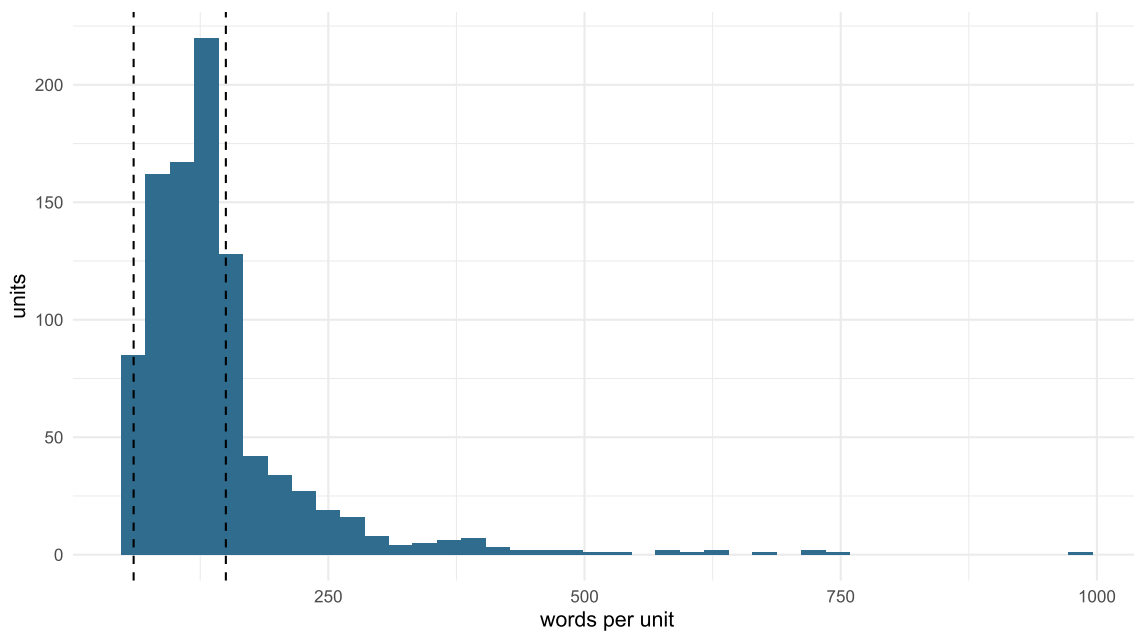


Figure 1: Words per kept unit. Units pile up between min_tokens and max_tokens (dashed); long author paragraphs sit to the right and are never split.

```
units |>
  dplyr::group_by(Year) |>
  dplyr::summarise(dropped = sum(n_tokens[!keep]) / sum(n_tokens)) |>
  ggplot(aes(Year, dropped)) +
```

```
geom_col(fill = viridis(1, begin = 0.6)) +
labs(x = NULL, y = "share of words dropped")
```

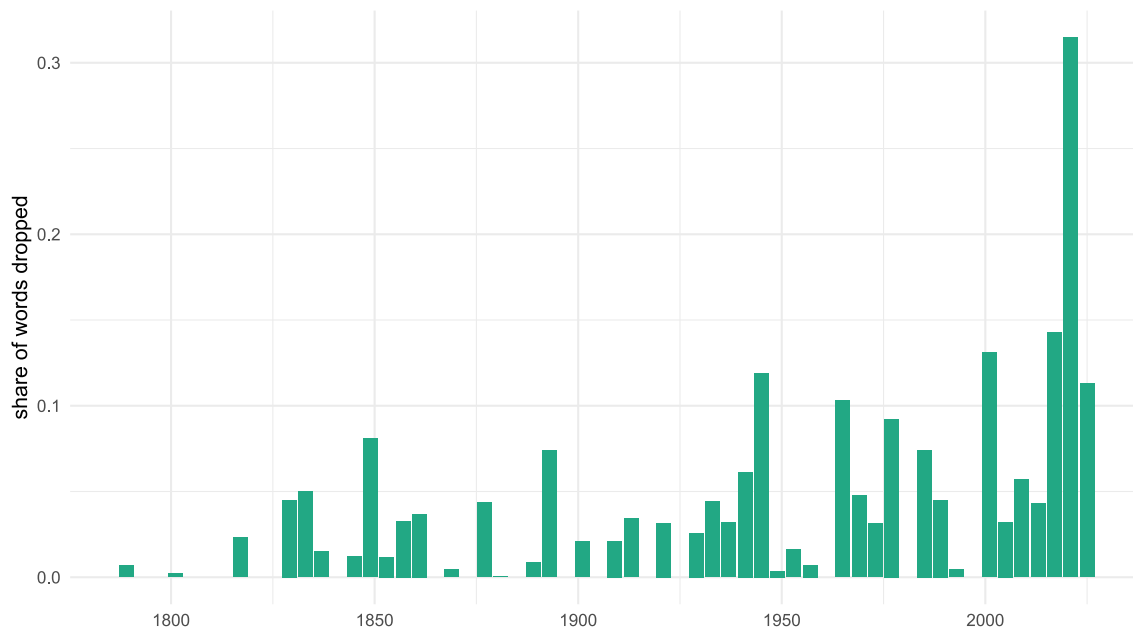


Figure 2: Share of words dropped by year. Dropping is not neutral if it concentrates in some years: those years then rest on longer, better-formed comments.

Handoff

```
readr::write_csv(dplyr::select(paragraphs, para_uid, atom_id, para_id, Year,
                               n_tokens, n_blocks, text), p_out("paragraphs.csv"))
readr::write_csv(dplyr::select(dropped, atom_id, Year, n_tokens, text),
                 p_out("dropped_units.csv"))
readr::write_csv(blocks, p_out("blocks.csv"))
saveRDS(list(min_tokens = tx$min_tokens, max_tokens = tx$max_tokens,
             min_sim = min_sim), p_out("merge_settings.rds"))
# one vector per kept paragraph, rows named by para_uid, for stance.qmd
vectors <- do.call(rbind, paragraphs$vec)
rownames(vectors) <- paragraphs$para_uid
saveRDS(vectors, p_out("paragraph_vectors.rds"))
writeLines(glue::glue("Wrote paragraphs.csv ({nrow(paragraphs)} units), \\
  dropped_units.csv ({nrow(dropped)}), blocks.csv, merge_settings.rds, \\
  paragraph_vectors.rds, embeddings.rds. Next: topics.qmd."))
```

Wrote paragraphs.csv (951 units), dropped_units.csv (170), blocks.csv, merge_settings.rds, paragraph_vectors.rds, embeddings.rds. Next: topics.qmd.